Étape 4 – Suivi et clôture de sprint

Revue & Rétrospective — Sprint 2 : Moteur ML

1. Revue du sprint — fonctionnalités terminées

- Un jeu de données d'entraînement (dataset.csv) a été constitué à partir de trafic réel : 34 exemples normaux et 5 exemples d'anomalies (scans de ports simulés avec nmap).
- La classe **FeatureExtractor** regroupe les paquets par IP source sur des fenêtres de temps et calcule les métriques résumées (paquets, octets, ports et IP distincts).
- Le modèle **Isolation Forest** a été entraîné et sauvegardé (model.joblib) ; il détecte 100% des anomalies du jeu de test après ajustement du seuil de décision.
- Les méthodes predict() et score_to_alert() de AnomalyDetector sont fonctionnelles.
- 5 tests unitaires (pytest) valident le comportement du détecteur.

Statut : toutes les tâches du Sprint 2 sont terminées, y compris la tâche Souhaitable (ajustement du seuil de détection).

2. Rétrospective

Ce qui a bien fonctionné

- Séparation claire entre capture de trafic normal (en0) et capture des scans (lo0), ce qui a permis d'obtenir un jeu de données propre et bien étiqueté.
- Le modèle détecte 100% des anomalies dès l'entraînement, avec une séparation nette des scores entre trafic normal et scans de ports.
- Les tests unitaires ont permis de confirmer rapidement le bon fonctionnement du détecteur.

Difficultés rencontrées

- Le scan sur localhost (127.0.0.1) n'était pas visible depuis l'interface Wi-Fi (en0) : il fallait utiliser l'interface loopback (lo0). Corrigé en adaptant la capture.
- Une capture continue sur lo0 regroupait tous les paquets du scan dans une seule fenêtre de temps, ne produisant qu'un seul exemple exploitable. Corrigé en répétant plusieurs captures courtes.
- Isolation Forest retourne des booléens NumPy plutôt que des booléens Python natifs, ce qui a fait échouer des comparaisons strictes (is True) dans les tests. Corrigé en comparant les valeurs plutôt que l'identité des objets.

Changements pour le prochain sprint

- Pour le Sprint 3, bien distinguer dès le départ les interfaces et sources de données utilisées (réseau réel vs localhost) avant d'écrire le code de capture.
- Prévoir des jeux de données un peu plus larges si le temps le permet, pour réduire encore le risque de faux positifs.